Voice Control Yolov11 Garbage Sorting

1. Function Description

After the program starts, voice commands are issued for garbage sorting. The program identifies garbage-labeled blocks in the image according to the instructions, calculates their positions in the world coordinate system, and then controls the robotic arm to grasp them. Based on the recognized garbage label names, they are classified and placed at the corresponding designated locations. At the same time, it will announce what type of garbage the grasped garbage-labeled block is.

2. Startup and Operation

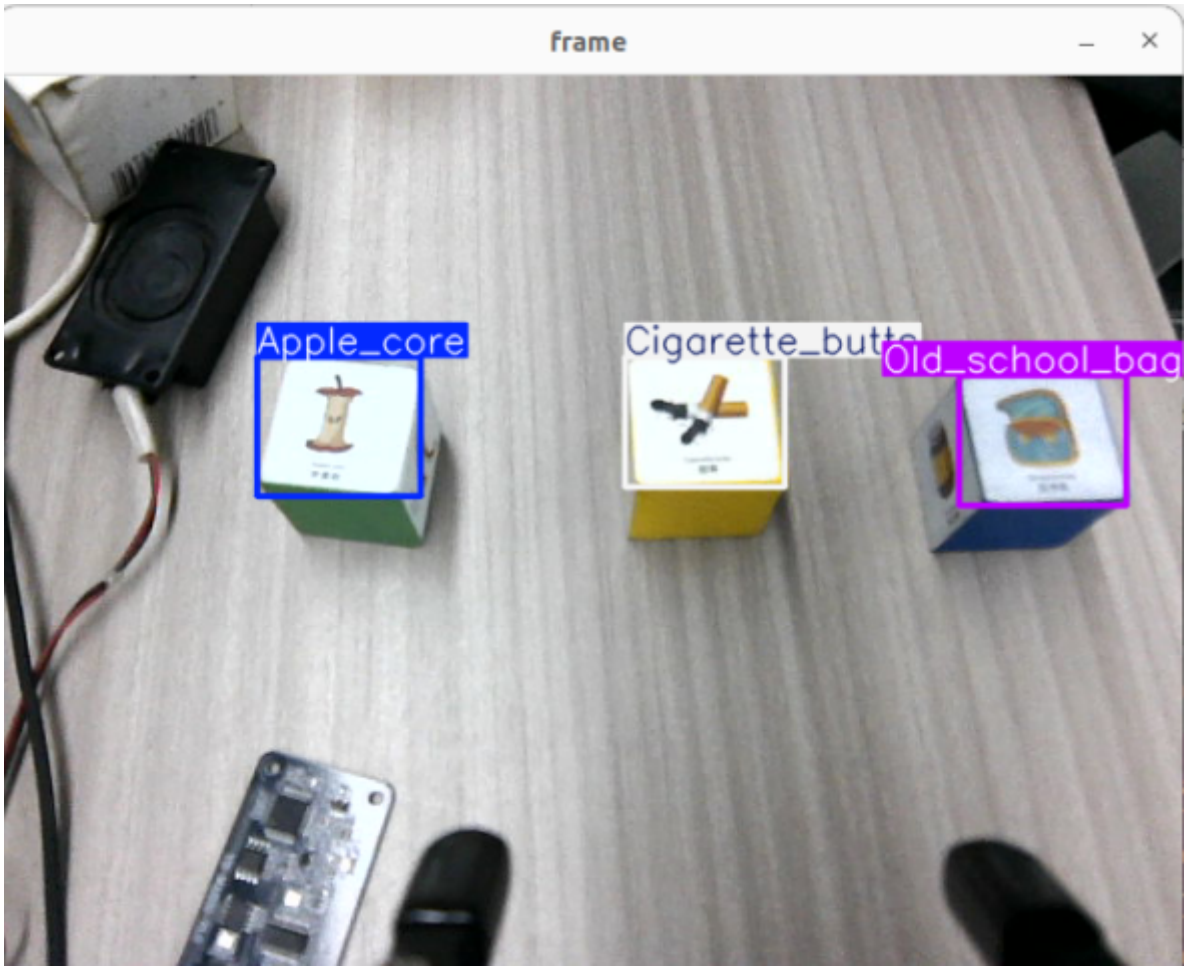
2.1 Startup

Orin motherboard users can directly open the terminal and enter the following commands.

```
# Start robotic arm + camera inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start voice recognition program
ros2 launch yahboom_speech speech.launch.py
# Start robotic arm control program
ros2 run dofbot_voice_ctrl grasp_VC
# Start garbage recognition program
ros2 run dofbot_voice_ctrl yolov11_garbage
```

2.2 Operation Steps

After all programs are running, place the blocks with garbage labels in the camera field of view. Say "Hi, yahboom" to the voice recognition module, and the speaker will announce "I am here". Then say "Start garbage sorting" to the voice module, and the voice module will reply "OK". The robotic arm will grasp the garbage block, then place it at the corresponding location. The voice module will announce what type of garbage it is. After placement is complete, it will announce "Placement complete", and the robotic arm returns to the recognition pose.



3. Core Code Analysis

Garbage Recognition Node yolov11_garbage

Source code path:
`/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/Yolov11/yolov11_garbage.py`

Import necessary libraries

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image
from message_filters import ApproximateTimeSynchronizer, Subscriber
from std_msgs.msg import Float32, Bool
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
import transforms3d as tfs
import tf_transformations as tf         # ROS2使用tf_transformations
import threading

from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
import pyzbar.pyzbar as pyzbar
from std_msgs.msg import Float32,Bool,Int16,Int8
```

```
import time
import queue
import math
from Arm_Lib import Arm_Device
from ultralytics import YOLO
import warnings
```

Initialize parameters

```
def __init__(self):
    # Initialize the ROS2 node with the name 'apriltag_detect'
    # 使用名称 'apriltag_detect' 初始化ROS2节点
    super().__init__('apriltag_detect')

    # Create an instance of the robotic arm device class
    # 创建机械臂设备类的实例
    self.Arm = Arm_Device()

    # Define initial joint angles for the robot arm
    # 定义机械臂的初始关节角度
    self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]

    # Message for publishing joint angle
    # 发布关节角度的消息
    self.joint5 = Int16()

    # Flag indicating whether garbage detection is active
    # 标志位，表示检测是否激活
    self.detect_flag = False

    self.compute_height = True

    # Index variable foridentification purposes
    # 识别目的的索引变量
    self.index = None

    # Create a service client for kinematics calculations
    # 创建用于运动学计算的客户端服务
    self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')

    # Publisher for setting joint angle
    # 设置关节角度的发布者
    self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)

    # Publisher for publishing position information
    # 位置信息的发布者
    self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)

    # Subscription to receive grasp completion status
    # 订阅以接收抓取完成状态
    self.subscription = self.create_subscription(
        Bool, 'grasp_done', self.GraspStatusCallback, qos_profile=1)

    # Bridge for converting ROS2 Image messages to OpenCV RGB format
    # 用于将ROS2 Image消息转换为OpenCV RGB格式的桥接器
    self.rgb_bridge = CvBridge()
```

```
        # Bridge for converting ROS2 Image messages to OpenCV depth format
        # 用于将ROS2 Image消息转换为OpenCV深度图格式的桥接器
        self.depth_bridge = CvBridge()

        # Flag for enabling position publishing
        # 启用位置发布的标志
        self.pubPos_flag = False

        # Previous time for FPS calculation
        # 用于FPS计算的前一时间点
        self.pr_time = time.time()

        # Initialize the AprilTag detector with specified parameters
        # 使用指定参数初始化AprilTag检测器
        self.at_detector = Detector(
            searchpath=['apriltags'],       # Search path for tag definitions
            families='tag36h11',           # Tag family type
            nthreads=8,                    # Number of threads for detection
            quad_decimate=2.0,             # Decimation factor for quad detection
            quad_sigma=0.0,                # Gaussian blur sigma for quad detection
            refine_edges=1,                # Edge refinement flag
            decode_sharpening=0.25,        # Sharpening factor for decoding
            debug=0                        # Debug level
        )

        # Target ID to detect
        # 要检测的目标ID
        self.target_id = 31

        # Lists to store center coordinates of detected tags
        # 存储检测到的标签中心坐标的列表
        self.Center_x_list = []
        self.Center_y_list = []

        # Move arm to initial position on startup
        # 启动时将机械臂移动到初始位置
        self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

        # Default distance parameter
        # 默认距离参数
        self.dist = 0.13

        # Subscription to receive raw camera images
        # 订阅接收原始相机图像
        self.subscription = self.create_subscription(
            Image, '/image_raw', self.ImageCallback, 1)

        # Flag to indicate if detection should start
        # 标志位，表示检测开始
        self.start_flag = True

        # Message for publishing play ID
        # 发布播放ID的消息
        self.play_id = Int8()

        # Classification lists for different types of waste
        # 不同类型垃圾的分类列表
```

```
    # Recyclable waste items
    # 可回收物
    self.recyclable_waste = ['Newspaper', 'Zip_top_can', 'Book',
'Old_school_bag']

    # Toxic/hazardous waste items
    # 有害垃圾
    self.toxic_waste = ['Syringe', 'Expired_cosmetics', 'Used_batteries',
'Expired_tablets']

    # Wet/organic waste items
    # 湿垃圾（厨余垃圾）
    self.wet_waste = ['Fish_bone', 'Egg_shell', 'Apple_core', 'Watermelon_rind']

    # Dry/other waste items
    # 干垃圾
    self.dry_waste = ['Toilet_paper', 'Peach_pit', 'Cigarette_butts',
'Disposable_chopsticks']

    # Subscription to receive voice recognition results
    # 订阅以接收语音识别结果
    self.sub_voice = self.create_subscription(
        Int8, "voice_result", self.getVoiceResultCallBack, 1)

    # Publisher for playing audio/voice feedback
    # 发布音频/语音反馈的发布者
    self.pub_playID = self.create_publisher(Int8, "player_id", qos_profile=10)
```

Image callback function

```
def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV RGB image format
    # 将ROS2 Image消息转换为OpenCV RGB图像格式
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Create copies of the image for processing and annotation
    # 创建图像的副本用于处理和标注
    result_image = np.copy(rgb_image)
    frame = np.copy(rgb_image)

    # Run YOLO model inference on the frame
    # 对帧运行YOLO模型推理
    results = model(frame)

    # Capture key press for user interaction
    # 捕获按键以进行用户交互
    key = cv2.waitKey(10)

    # Spacebar enables position publishing
    # 空格键启用位置发布
    if key == 32:
        self.pubPos_flag = True

    # Extract bounding boxes from detection results
    # 从检测结果中提取边界框
    boxes = results[0].boxes
```

```
    # Process detections if boxes exist and detection is enabled
    # 如果边界框存在且检测已启用，则处理检测结果
    if boxes != [None] and self.start_flag == True:
        for box in boxes:  # Iterate through detections per image
            # Extract bounding box coordinates
            # 提取边界框坐标
            x_min, y_min, x_max, y_max = map(int, box.xyxy[0])

            # Get class ID and confidence score
            # 获取类别ID和置信度分数
            class_id = int(box.cls)
            confidence = float(box.conf)

            # Create label string with class name and confidence
            # 创建包含类别名称和置信度的标签字符串
            label = f"{model.names[class_id]} {confidence:.2f}"
            print("model.names[class_id]: ", model.names[class_id])

            # Calculate center position of the detected object
            # 计算检测到的物体中心位置
            center_x = (x_min + x_max) // 2
            center_y = (y_min + y_max) // 2

            # Normalize coordinates
            # 将坐标归一化
            # a: horizontal offset (normalized)
            # a：水平偏移（归一化）
            # b: vertical offset with scaling factor (normalized)
            # b：带缩放因子的垂直偏移（归一化）
            (a, b) = (round(((320 - center_x) / 4000), 5), round(((480 -
center_y) / 3000) * 0.8 + 0.13, 5))

            # If position publishing is enabled
            # 如果位置发布已启用
            if self.pubPos_flag == True:
                self.pubPos_flag = False  # Disable after publishing

                # Create AprilTagInfo message with position data
                # 创建包含位置数据的AprilTagInfo消息
                center = AprilTagInfo()
                center.x = b
                center.y = -a
                center.z = 0.03

                # Store the detected object name
                # 存储检测到的物体名称
                self.name = str(model.names[class_id])

                # Classify waste type and assign corresponding ID
                # 分类垃圾类型并分配对应的ID

                # Recyclable waste (id = 1)
                # 可回收物（id = 1）
                if str(model.names[class_id]) in self.recyclable_waste:
                    center.id = 1
                    if self.name == "Newspaper":
                        self.play_id.data = 96
```

```
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Zip_top_can":
                    self.play_id.data = 94
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Book":
                    self.play_id.data = 97
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Old_school_bag":
                    self.play_id.data = 95
                    self.pub_playID.publish(self.play_id)

            # Toxic/hazardous waste (id = 3)
            # 有害垃圾（id = 3）
            elif str(model.names[class_id]) in self.toxic_waste:
                center.id = 3
                if self.name == "Syringe":
                    self.play_id.data = 98
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Expired_cosmetics":
                    self.play_id.data = 100
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Expired_tablets":
                    self.play_id.data = 101
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Used_batteries":
                    self.play_id.data = 99
                    self.pub_playID.publish(self.play_id)

            # Wet/organic waste (id = 2)
            # 湿垃圾（厨余垃圾）（id = 2）
            elif str(model.names[class_id]) in self.wet_waste:
                center.id = 2
                if self.name == "Fish_bone":
                    self.play_id.data = 102
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Watermelon_rind":
                    self.play_id.data = 103
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Apple_core":
                    self.play_id.data = 104
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Egg_shell":
                    self.play_id.data = 105
                    self.pub_playID.publish(self.play_id)

            # Dry(id = 4)
            # 干垃圾（id = 4）
            elif str(model.names[class_id]) in self.dry_waste:
                center.id = 4
                if self.name == "Toilet_paper":
                    self.play_id.data = 109
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Disposable_chopsticks":
                    self.play_id.data = 106
                    self.pub_playID.publish(self.play_id)
                elif self.name == "Cigarette_butts":
                    self.play_id.data = 107
                    self.pub_playID.publish(self.play_id)
```

```
                    elif self.name == "Peach_pit":
                        self.play_id.data = 108
                        self.pub_playID.publish(self.play_id)

                # Print and publish the position info
                # 打印并发布位置信息
                print("center: ", center)
                self.pos_info_pub.publish(center)

    # Annotate the frame with detection results
    # 用检测结果标注帧
    annotated_frame = results[0].plot()

    # Convert RGB to BGR for OpenCV display
    # 将RGB转换为BGR以进行OpenCV显示
    annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_RGB2BGR)

    # Display the annotated frame
    # 显示标注后的帧
    cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

    # Capture key press for window events
    # 捕获按键以处理窗口事件
    key = cv2.waitKey(1)
```